CAPSTONE PROJECT

HomeLab Sentinel

Requirements & Design Document

Simulated Server Monitoring, Security Event Analysis, and AI-Assisted Troubleshooting

Prepared for: IT Capstone Course

Date: October 2026

Project repository: https://github.com/des0620/HomeLab-Sentinel

Document status: Implementation-aligned design baseline; review before submission.

Document Scope

HomeLab Sentinel is a working Flask-based prototype that demonstrates home-server monitoring workflows using simulated resource metrics, service failures, application logs, and security events. It stores demo state and records in SQLite and provides knowledge-base-grounded troubleshooting with optional OpenAI assistance. It does not connect to or continuously monitor a real server or network.

This document describes the implemented demonstration system. Numeric performance targets are proposed acceptance criteria, not measured benchmark results. The SQLite table definitions in Section 4 were verified against the supplied app.py CREATE TABLE statements.

1. Functional Requirements

User role: a home-lab operator or evaluator interacting with the local browser dashboard. All events in the current MVP are simulated.

ID	Requirement / user story	Acceptance evidence
FR-01	As an operator, I can view simulated CPU, memory, and storage utilization.	Dashboard presents percentage metrics.
FR-02	As an operator, I can view the status of simulated Plex, Sonarr, and Radarr services.	Service statuses visible on dashboard.
FR-03	As an operator, I can trigger high CPU, disk warning, and Docker failure scenarios.	Simulation action changes corresponding state and alert.
FR-04	As an operator, I can reset simulated server state.	Metrics/services return to baseline.
FR-05	As an operator, I can generate sample application/system logs.	Plex, Docker, and disk log events appear in event history.
FR-06	As an operator, I can simulate security activity.	Failed SSH, network activity, and unauthorized access events recorded.
FR-07	As an operator, I can see rule-based security findings.	Configured event patterns produce visible findings.
FR-08	As an operator, I can reset security events without resetting server state.	Security state cleared; server state preserved.
FR-09	As an operator, I can view event logs and alert history.	Recent records shown in dashboard tables.
FR-10	As an operator, I can request troubleshooting guidance for an active issue.	Relevant local runbook and AI/fallback guidance shown.
FR-11	As an operator, I can use the	Responsive layout stacks panels

	dashboard on desktop or smaller displays.	on narrow screens.
FR-12	As an evaluator, I can run automated regression tests.	Existing test suites execute successfully.

2. Non-Functional Requirements

ID	Category	Design requirement / proposed acceptance criterion
NFR-01	Usability	Server and security overview panels appear side by side at desktop widths; controls are clearly labeled.
NFR-02	Responsiveness	Dashboard adapts to mobile widths without hiding core functionality.
NFR-03	Performance	Target: ordinary local dashboard interactions complete within 2 seconds, excluding external AI latency; measure before asserting compliance.
NFR-04	Security	OpenAI credentials are supplied through local environment configuration, not committed to version control.
NFR-05	Output safety	AI-generated Markdown is sanitized before display to reduce unsafe HTML/script rendering.
NFR-06	Reliability	Local knowledge-base guidance remains available when an external AI response cannot be obtained.
NFR-07	Data integrity	SQLite retains demo state and records across normal page refreshes; resets affect their intended scope.
NFR-08	Maintainability	Python modules, Flask templates, troubleshooting knowledge, and automated tests remain version controlled.

NFR-09	Transparency	Interface and documentation explicitly identify simulated data and non-production limitations.
NFR-10	Testability	Regression suite covers core server, troubleshooting, and security workflows; all 26 tests passed in latest reported run.

3. System Architecture

The browser submits actions to the local Flask application. Flask generates demo data, applies detection logic, reads/writes SQLite records, retrieves local troubleshooting guidance, and optionally calls OpenAI. The browser receives rendered dashboard and troubleshooting pages.

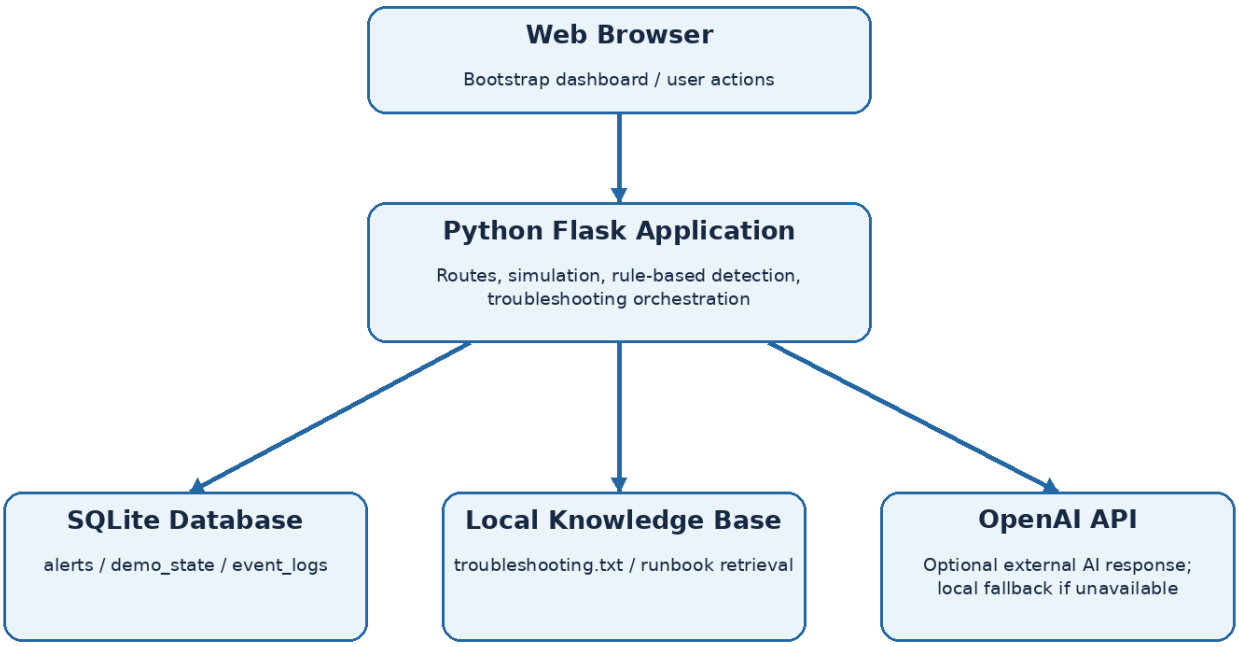


Figure 1. Logical architecture of the implemented prototype. Arrows indicate primary interactions, not an exhaustive sequence diagram.

3.1 Major Components

- Web interface: Bootstrap-based dashboard with monitoring summaries, simulation controls, security findings, event logs, and alert history.
- Flask application (app.py): HTTP routes, simulated events, alert management, detection rules, and troubleshooting flow.
- SQLite: local persistent store for alert records, demo state, and event logs.
- Local troubleshooting knowledge base: text runbook used to ground guidance and support fallback behavior.

- OpenAI integration: optional remote language-model assistance using a locally configured API key.

3.2 Event and Troubleshooting Flow

An operator activates a simulation control; Flask updates demo state and/or inserts event records. The dashboard displays the resulting metrics, statuses, counters, and findings. For troubleshooting, the operator opens a category-specific guidance page; the application uses the local runbook and may request an AI-generated response. The system remains a demonstration rather than a live telemetry collector.

4. Database Design

The application uses three SQLite tables. The physical column definitions below were verified against app.py. SQLite CURRENT_TIMESTAMP provides a default UTC timestamp as text. The schema does not define foreign keys.

Table	Purpose	Known logical content
alerts	Historical alert records	id INTEGER PRIMARY KEY AUTOINCREMENT; created_at TEXT DEFAULT CURRENT_TIMESTAMP; category TEXT NOT NULL; message TEXT NOT NULL
demo_state	Simulated server settings	name TEXT PRIMARY KEY; value TEXT NOT NULL. Default values inserted with INSERT OR IGNORE; CPU, memory, and storage converted to integers when read.
event_logs	Simulated application and security events	id INTEGER PRIMARY KEY AUTOINCREMENT; created_at TEXT DEFAULT CURRENT_TIMESTAMP; source TEXT NOT NULL; severity TEXT NOT NULL; category TEXT NOT NULL; message TEXT NOT NULL

4.1 Logical Relationships

Security detection rules (rolling five-minute window): 3 Failed SSH Logins event batches → Possible Brute-Force Activity (CRITICAL); 3 Suspicious Network Activity events → Repeated Suspicious Network Activity (WARNING); 1 Unauthorized Access Attempt → Unauthorized Access Detected (CRITICAL). These are simulated, rule-based findings, not confirmed live attacks.

The three tables are independent at the database level (no declared foreign keys). Flask application logic reads demo_state for current simulated resource and service values, stores historical alerts in

alerts, and reads event_logs for event history and five-minute security pattern analysis. Security findings are calculated from event_logs rather than stored in a separate findings table. The security reset clears relevant security logs and associated alert history without changing server state or unrelated logs.

4.2 Data Lifecycle

- Creation: simulation routes generate sample metrics, state transitions, and log records.
- Read: dashboard routes retrieve current state, event history, and active findings.
- Reset: server reset and security reset have different scopes.
- Storage: local SQLite file is runtime data and should not be treated as a shared production monitoring database.

5. User Interface Wireframe

The current desktop dashboard emphasizes immediate comparison: server monitoring on the left and security monitoring on the right. Simulation controls follow the overview, with detailed event logs and alert history below. At smaller widths, the two panels stack vertically.

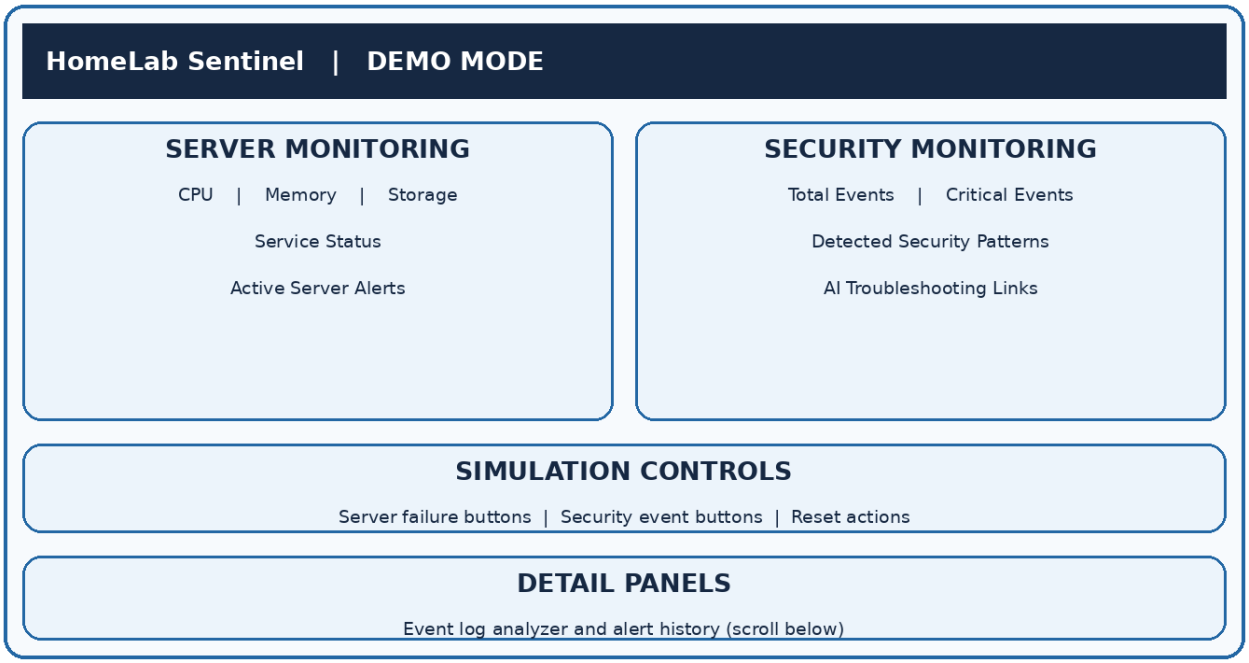


Figure 2. Conceptual wireframe of the current side-by-side dashboard layout.

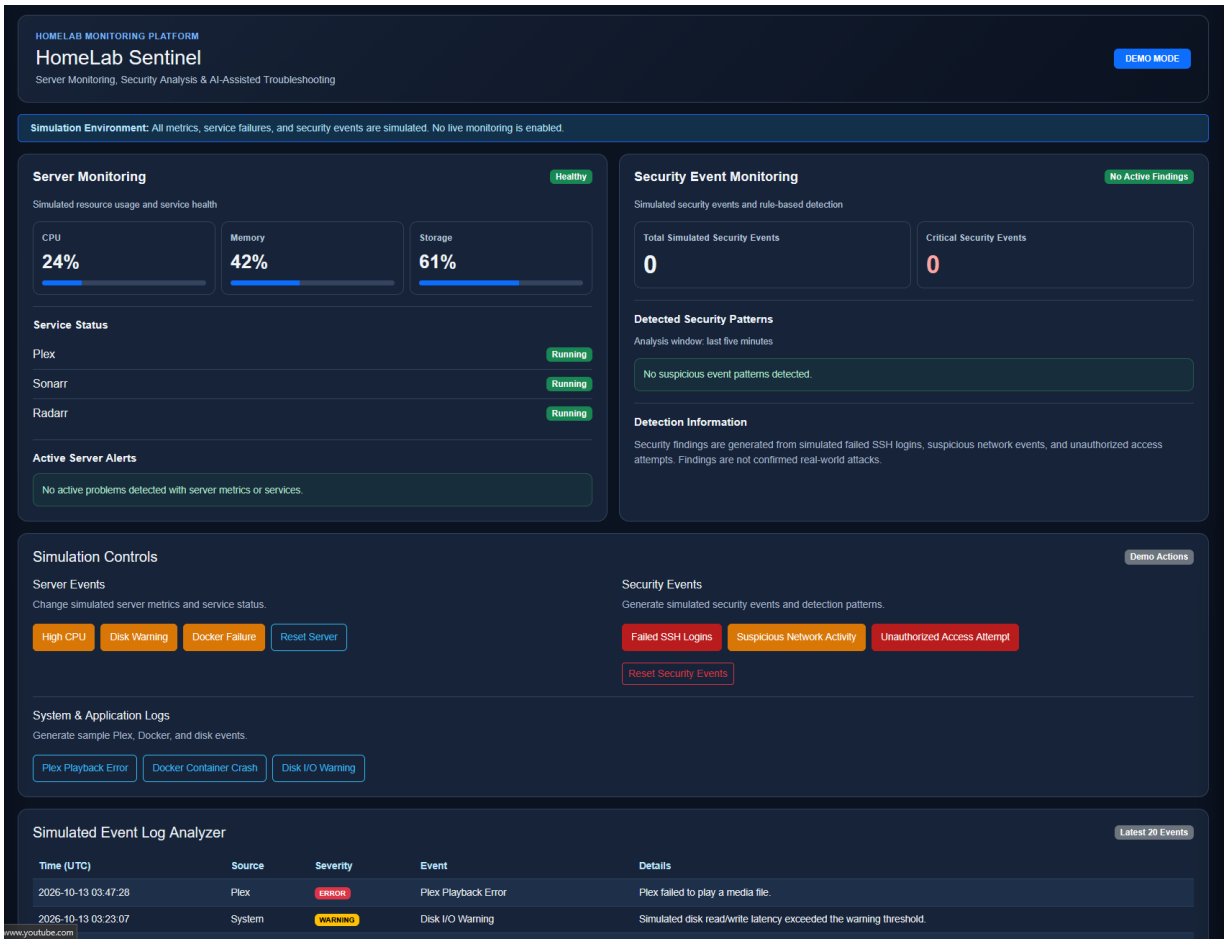


Figure 3. Implemented HomeLab Sentinel dashboard showing side-by-side server and security monitoring, simulation controls, and event logging.

5.1 Implemented Feature Evidence

The following screenshots supplement the conceptual wireframe by demonstrating implemented behavior. They document simulated detections and advisory troubleshooting; they are not evidence of monitoring live infrastructure.

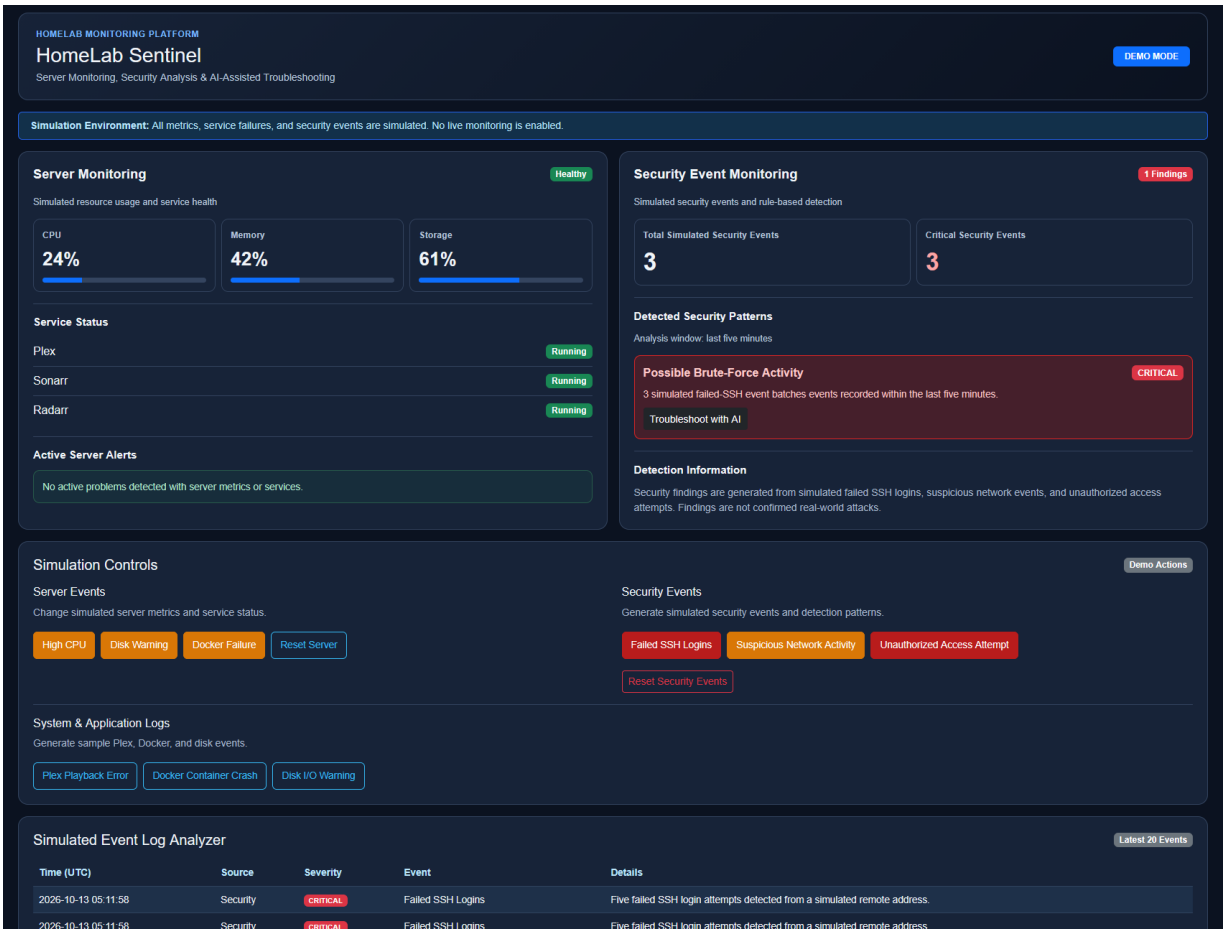


Figure 4. Simulated brute-force detection after three failed-SSH event batches within five minutes; the dashboard displays a CRITICAL finding.

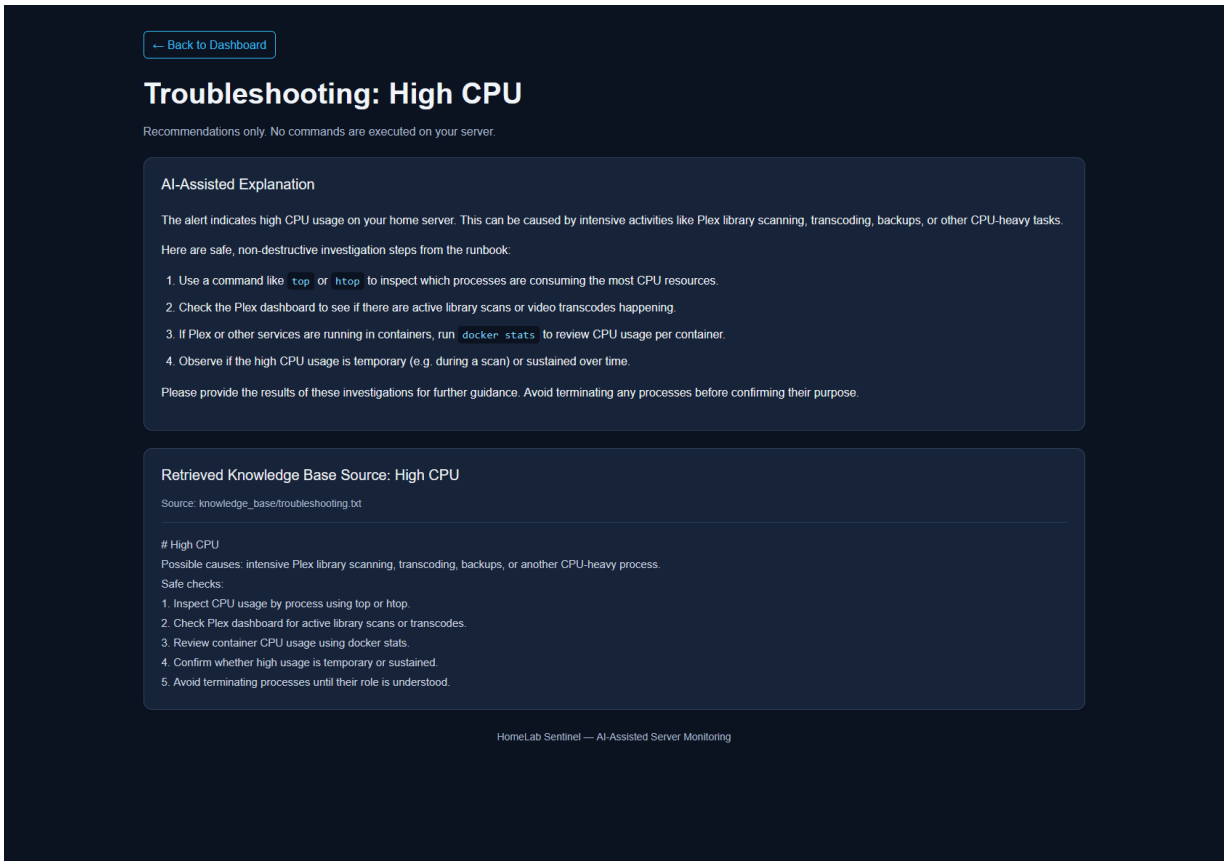


Figure 5. High CPU troubleshooting view showing AI-assisted recommendations and the retrieved local knowledge-base source.

5.2 Main Interaction Paths

- Server incident: select High CPU, Disk Warning, or Docker Failure → inspect updated server panel → open troubleshooting.
- Security incident: generate repeated failed SSH or suspicious network events, or unauthorized access → inspect security findings → open troubleshooting.
- Reset: use separate Server Reset and Security Reset actions to demonstrate independent scopes.
- Audit trail: review recent event logs and alert history beneath the primary monitoring panels.

6. Test Plan Outline

Test approach: automated Python regression tests for Flask/server behavior, local troubleshooting, and security detection, supplemented by manual browser-based usability and demo checks.

Design-to-test traceability: server monitoring and alert/reset workflows are covered by test_phase2.py; knowledge-base retrieval and troubleshooting behavior by test_phase3.py; and simulated security logging, detection thresholds, reset isolation, and associated dashboard behavior by test_phase5.py. The 26-test total is the latest reported successful run, not an independently reproduced result.

Security acceptance scenarios: three Failed SSH Logins event batches within five minutes produce a CRITICAL Possible Brute-Force Activity finding; three Suspicious Network Activity events produce a

WARNING finding; one Unauthorized Access Attempt produces a CRITICAL finding. Security reset should clear security events/findings while preserving server state and nonsecurity event records.

Test area	Representative scenarios	Method / status
Server state	High CPU, disk warning, Docker failure, server reset	test_phase2.py — 4 automated tests reported passing
Troubleshooting	Runbook retrieval and fallback behavior	test_phase3.py — 4 automated tests reported passing
Security events	Event generation, thresholds, findings, reset isolation	test_phase5.py — 18 automated tests reported passing
Dashboard usability	Two-column display, readable cards, responsive stacking	Manual visual review; user approved desktop layout
AI integration	Valid API response and unavailable API fallback	Manual verification recommended before submission
Repository hygiene	No tracked .env or virtual environment at checked paths	Git check completed for specified paths

6.1 Test Execution

From the activated project virtual environment, execute:

python -m unittest -v test_phase2 test_phase3 test_phase5

Latest reported outcome: 26 tests passed. This is a user-reported execution result, not a benchmark or independent re-run within this document.

6.2 Manual Acceptance Checklist

- Load dashboard in a browser; verify both monitoring panels are visible on a desktop viewport.
- Simulate a high CPU condition and verify an alert and relevant troubleshooting path.
- Reset server state and verify expected baseline.
- Trigger repeated failed SSH events within the configured five-minute window; verify a detection finding.
- Trigger unauthorized access; verify a security finding.
- Reset security events and verify server state and nonsecurity logs remain unchanged.
- Inspect event history and alert history for expected entries.
- Confirm API key is not displayed or committed; test AI fallback when the API is unavailable.
- Check a narrow browser viewport for stacked, readable panels.

7. Constraints, Assumptions, and Future Work

Current constraints: metrics and events are simulated; the application is a local prototype; security rules are illustrative and not a production intrusion detection system; OpenAI calls require network access and configured credentials; performance and load limits have not been formally benchmarked.

Potential future improvements: integrate actual Unraid metrics and service health, collect authenticated host logs, add configurable detection thresholds, provide user authentication, add deployment hardening and audit controls, and measure response times under load. These are future enhancements, not current functional claims.

8. Traceability Summary

The six required deliverables are covered as follows: functional requirements in Section 1; non-functional requirements in Section 2; architecture diagram in Section 3; database design in Section 4; UI wireframe in Section 5; and test plan in Section 6.

Appendix A. Repository and Implementation References

GitHub: https://github.com/des0620/HomeLab-Sentinel

Key artifacts: app.py; templates/dashboard.html; knowledge_base/troubleshooting.txt; test_phase2.py; test_phase3.py; test_phase5.py; README.md.

Implementation verification: database table definitions were checked against supplied app.py excerpts. Figures 3–5 document the implemented dashboard and selected feature behavior. Confirm the final test run and repository state before submission.